

UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO

Facultad de Ciencias



MANEJO DE DATOS

Investigación

Equipo: El real maíz

Integrantes:

Gómez Avilés Danae

Padilla Benítez Zuemi

Alejandra

González González

Jennifer

Sánchez Nutes Sakti

Sarai

Objetivo:

El alumno pondrá en práctica la resolución de problemas aplicando buenas prácticas de programación y principios SOLID con Python, incluyendo validación robusta de datos, manejo de excepciones, análisis de complejidad algorítmica y comparación empírica de algoritmos.

Parte A- PEP8

MALO- Antes

BUENO- Despues

De acuerdo con la guía de estilo, las 10 reglas fundamentales para escribir código limpio en Python, son:

1.-**Indentación:** Usa siempre 4 espacios por nivel, nunca tabuladores.

- Malo: *def funcion():\n return True*
- Bueno: *def funcion():\n return True*

2.-**Longitud de línea:** Limita las líneas a un máximo de 79 caracteres para facilitar la lectura.

- Malo: *print("Esta es una línea exageradamente larga que no se puede leer bien en monitores pequeños o divididos.")*
- Bueno: *print("Esta es una línea corta y "\n "fácil de leer.")*

3.-**Importaciones separadas:** Cada módulo debe importarse en una línea distinta.

- Malo: *import sys, os*
- Bueno: *import os\nimport sys*

4.-**Nombres de clases:** Deben usar la convención CapWords (PascalCase).

- Malo: *class mi_clase_principal:*
- Bueno: *class MiClasePrincipal:*

5.-**Nombres de funciones y variables:** Deben estar en minúsculas y separadas por guiones bajos (snake_case).

- Malo: *def CalcularTotal(MiVariable):*
- Bueno: *def calcular_total(mi_variable):*

Parte A- PEP8

6.-Espacios en operadores: Usa un solo espacio alrededor de operadores de asignación o comparación.

- Malo: `x=5+3`
- Bueno: `x = 5 + 3`

7.-Espacios en paréntesis: No dejes espacios extra justo adentro de los paréntesis, corchetes o llaves.

- Malo: `lista( 1, 2, 3 )`
- Bueno: `lista(1, 2, 3)`

8.-Líneas en blanco: Usa dos líneas en blanco para separar clases y funciones principales.

- Malo: `class A:\n pass\n class B:\n pass`
- Bueno: `class A:\n pass\n\n\nclass B:\n pass`

9.-Nombres de constantes: Deben ir completamente en mayúsculas y separadas por guiones bajos.

- Malo: `limite_maximo = 100`
- Bueno: `LIMITE_MAXIMO = 100`

10.-Comentarios de bloque: Deben ser oraciones completas, empezar con mayúscula y usar `#` seguido de un espacio.

- Malo: `#esto calcula la edad`
- Bueno: `# Calcula la edad del usuario basándose en su año de nacimiento.`

PARTE B-¿QUÉ ES EL SÓLID?

Es un conjunto de cinco reglas de diseño en la programación que te ayudan a escribir código ordenado. Su objetivo principal es evitar que tu programa se vuelva un "castillo de naipes" donde, si necesitas agregar una función nueva o hacer un cambio, todo lo demás se derrumbe.

1.S - Single Responsibility Principle (Principio de responsabilidad única)

- Problema que resuelve: Evita el código donde una sola clase hace de todo, lo que provoca que si modificas un pequeño detalle, corras el riesgo de romper otras funciones totalmente desconectadas.
- Imagina que tienes una clase para calcular la prima de un seguro estudiantil. Su único trabajo en la vida debe ser hacer ese cálculo matemático. Si a esa misma clase le agregas funciones para mandar correos electrónicos, le estás dando más tareas de las que le tocan. Cada clase debe dedicarse a una sola cosa.

1. Código "malo":

```
class SeguroEstudiantil:
    def calcular_prima(self, edad):
        return edad * 150

    def enviar_correo_alumno(self):
        print("Enviando recibo por correo...")oliza)liza):
        self.db.guardar(poliza)
```

PARTE B-¿QUÉ ES EL SÓLID?

2. Código "bueno":

```
class SeguroEstudiantil:
    def calcular_prima(self, edad):
        return edad * 150

class NotificadorCorreo:
    def enviar_correo(self, mensaje):
        print(f"Enviando: {mensaje}")
```

2. O - Open/Closed Principle (Principio de abierto/cerrado)

- Problema que resuelve: Evita tener que reescribir código que ya funciona (y que ya probaste exhaustivamente) solo porque necesitas agregar una nueva categoría o característica al sistema.
- Tu código debe estar abierto para crecer, pero cerrado para ser modificado. Piensa en esto como ponerle un accesorio a un cachorro: le puedes añadir una placa o un suéter nuevo (extensión) sin tener que alterar al perrito en sí (modificación). Si te la pasas agregando cada vez que hay un nuevo tipo de dato, estás rompiendo esta regla.if/elif

PARTE B-¿QUÉ ES EL SÓLID?

1. Código "malo":

```
class CalculadoraDescuentos:
    def aplicar_descuento(self, tipo_seguro, precio):
        if tipo_seguro == "Vida":
            return precio * 0.90
        elif tipo_seguro == "Auto":
            return precio * 0.85
        # Si mañana agregas seguro de "Gastos Médicos", debes modificar esta clase.
```

2. Código "bueno":

```
from abc import ABC, abstractmethod

class Seguro(ABC):
    @abstractmethod
    def aplicar_descuento(self, precio):
        pass

class SeguroVida(Seguro):
    def aplicar_descuento(self, precio):
        return precio * 0.90

class SeguroAuto(Seguro):
    def aplicar_descuento(self, precio):
        return precio * 0.85
```

PARTE B-¿QUÉ ES EL SÓLID?

3. L - Liskov Substitution Principle (Principio de sustitución de Liskov)

- Problema que resuelve: Previene errores y caídas del sistema cuando usas clases "hijas" que no se comportan como la clase "padre" prometió.
- Si tienes una clase principal y creas subclases a partir de ella, deberías poder usar cualquiera de las subclases sin que tu programa truene. Las hijas deben cumplir todas las expectativas de la clase padre. Si el padre dice "todos hacemos sonido", y una hija lanza un error al intentar hacerlo, el programa fallará.

1. Código "malo":

```
class Mascota:
    def ladrar(self):
        pass

class Cachorro(Mascota):
    def ladrar(self):
        print("¡Guau!")

class Gato(Mascota):
    def ladrar(self):
        raise Exception("Los gatos no ladran") # Rompe la expectativa de la clase padre
```

PARTE B-¿QUÉ ES EL SÓLID?

2. Código "bueno":

```
class Mascota:
    def hacer_sonido(self):
        pass

class Cachorro(Mascota):
    def hacer_sonido(self):
        print("¡Guau!")

class Gato(Mascota):
    def hacer_sonido(self):
        print("¡Miau!")
```

4.I - Interface Segregation Principle (Principio de segregación de interfaces)

- **Problema que resuelve:** Evita obligar a las clases a depender de métodos o funciones que nunca van a utilizar.
- Es mejor tener herramientas pequeñas y específicas que una sola herramienta gigante que haga de todo. Si vas a un restaurante, no quieres que te obliguen a pedir un menú de 5 tiempos si solo querías un café. No obligues a tus clases a incluir métodos vacíos solo por cumplir con un diseño genérico.

PARTE B-¿QUÉ ES EL SÓLID?

1. Código "malo":

```
from abc import ABC, abstractmethod

class Trabajador(ABC):
    @abstractmethod
    def programar(self): pass

    @abstractmethod
    def calcular_probabilidades(self): pass

class Actuario(Trabajador):
    def programar(self):
        print("Escribiendo código en Python")
    def calcular_probabilidades(self):
        print("Calculando riesgo")

class Programador(Trabajador):
    def programar(self):
        print("Escribiendo código")
    def calcular_probabilidades(self):
        pass # Obligado a tener este método aunque su trabajo no sea la probabilidad
```

2. Código "bueno":

```
class Programable(ABC):
    @abstractmethod
    def programar(self): pass

class Calculador(ABC):
    @abstractmethod
    def calcular_probabilidades(self): pass

class Actuario(Programable, Calculador):
    def programar(self): print("Escribiendo código")
    def calcular_probabilidades(self): print("Calculando riesgo")

class Programador(Programable):
    def programar(self): print("Escribiendo código")
```

PARTE B-¿QUÉ ES EL SÓLID?

5. D - Dependency Inversion Principle (Principio de inversión de dependencias)

- **Problema que resuelve:** Reduce el acoplamiento entre la lógica importante de tu programa y las herramientas externas, facilitando el cambio de tecnologías (como bases de datos o servidores) en el futuro.
- Tu programa principal no debe conectarse directamente a detalles específicos, sino a "enchufes" genéricos. Por ejemplo, si tu simulador guarda datos, no debe conectarse directamente a "MySQL"; debe conectarse a un concepto abstracto de "Base De Datos". Así, si mañana decides guardar los datos en un archivo de Excel, el núcleo de tu programa ni se entera del cambio.

1. Código "malo":

```
○○○  
  
class BaseDeDatosMySQL:  
    def guardar(self, datos):  
        print("Guardando en MySQL")  
  
class SistemaSeguros:  
    def __init__(self):  
        self.db = BaseDeDatosMySQL() # Dependencia directa a una herramienta específica  
  
    def registrar_poliza(self, poliza):  
        self.db.guardar(poliza)
```

PARTE B-¿QUÉ ES EL SÓLID?

2. Código "bueno":

```
class Repositorio(ABC):
    @abstractmethod
    def guardar(self, datos): pass

class BaseDeDatosMySQL(Repositorio):
    def guardar(self, datos):
        print("Guardando en MySQL")

class SistemaSeguros:
    def __init__(self, repositorio: Repositorio):
        self.repositorio = repositorio # Depende de un concepto general, no de MySQL

    def registrar_poliza(self, poliza):
        self.repositorio.guardar(poliza)
        self.db.guardar(poliza)
```

BIBLIOGRAFÍA

- GeeksforGeeks. (2023, 15 de marzo). Liskov substitution principle in Python. <https://www.geeksforgeeks.org/liskov-substitution-principle-in-python/>
- DigitalOcean. (2021, 10 de septiembre). SOLID: The first 5 principles of object-oriented design. <https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design>
- Python Engineer. (2021, 12 de abril). SOLID principles in Python: Dependency inversion principle. <https://www.python-engineer.com/posts/solid-principles-python/>